

RL Bot Supplementary Walkthrough

This supplement covers only the reinforcement-learning bot work in the marble racer project: the training environment, PPO setup, artifact pipeline, checkpointed results, and the current deployment status of the exported policy.

Artifact slice	Overnight run dated 2026-04-05, checkpoint generations 17 through 27
Observed total timesteps	2,924,544 by the latest recorded checkpoint
Recorded checkpoint outcomes	5 checkpoints, 7 finishes, 2 off-track, 1 stall, 1 timeout
Best successful checkpoint	Generation 27 with reward 116.63 and finish time 34.92 s
Fastest successful finish	Generation 22 in 31.37 s
Runtime export status	Exported policy shape is 23 -> 64 -> 64 -> 2; runtime compatibility flag = False

Pipeline

the marble bot project

Stage 2

- Track: default full course
- Obstacles and boosts enabled

and finish spawn curriculum at 0, 18, 36, 54, 68, 82, 96, 110 m
ation •> 2 checkpoints every 100k steps with replay snapshot

y head is 2-D, while the residual runtime controller expects 3 out

The RL work used a direct-control training environment. The observation vector contains 23 normalized features grouped around progress and velocity, lane error and contact state, boost and obstacle context, and near-term track geometry. The PPO policy therefore learned to steer and brake directly in simulation rather than only nudging a scripted planner.

Method

The code implements a two-stage curriculum in `scripts/train_bot_policy.py`. Stage 1 trains on the short time-trial course without obstacles to stabilize basic motion. Stage 2 moves to the full default track with boosts and moving hazards, and resets spawn positions along a curriculum of distances from 0 m through 110 m so the policy repeatedly rehearses late-course sections instead of always relearning the opening ramp.

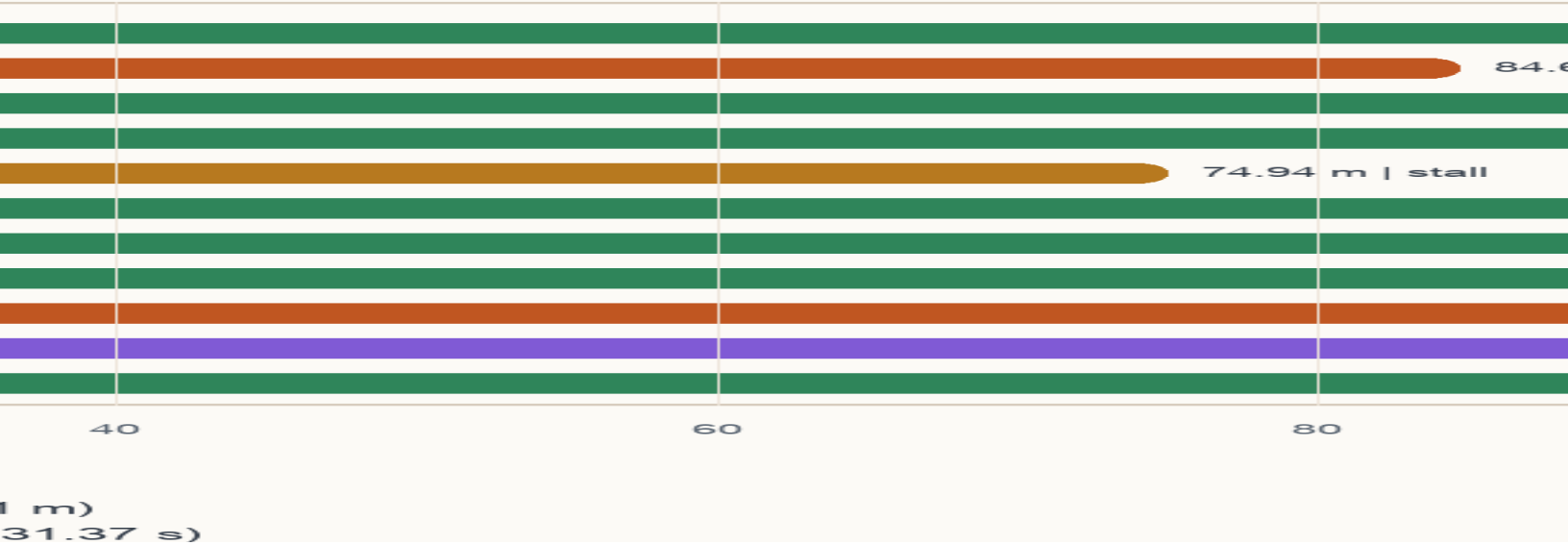
The PPO configuration is consistent and compact: an MLP policy with hidden layers [64, 64], `n_steps = 1024`, `batch_size = 256`, `gamma = 0.995`, `gae_lambda = 0.95`, entropy coefficient = 0.01, learning rate = $3e-4$, and clip range = 0.2. Checkpoints are saved every 100k timesteps, after which the code runs an evaluation rollout and refreshes the live monitor snapshot assets.

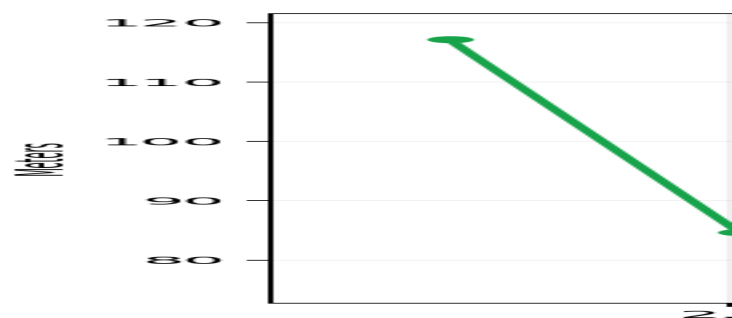
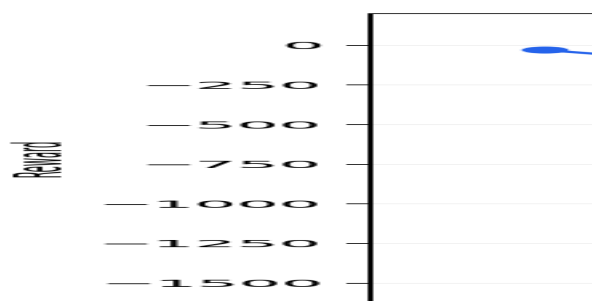
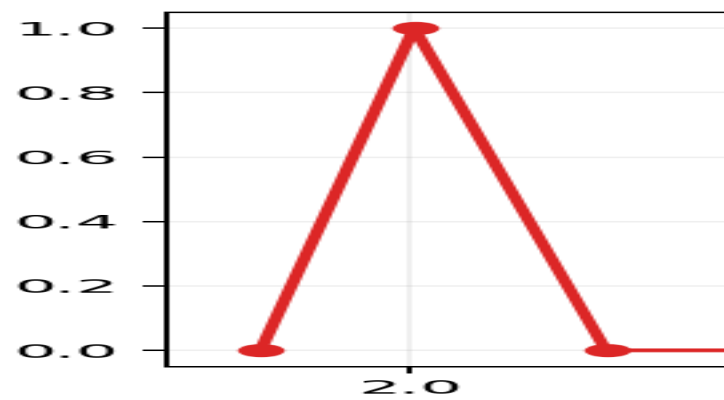
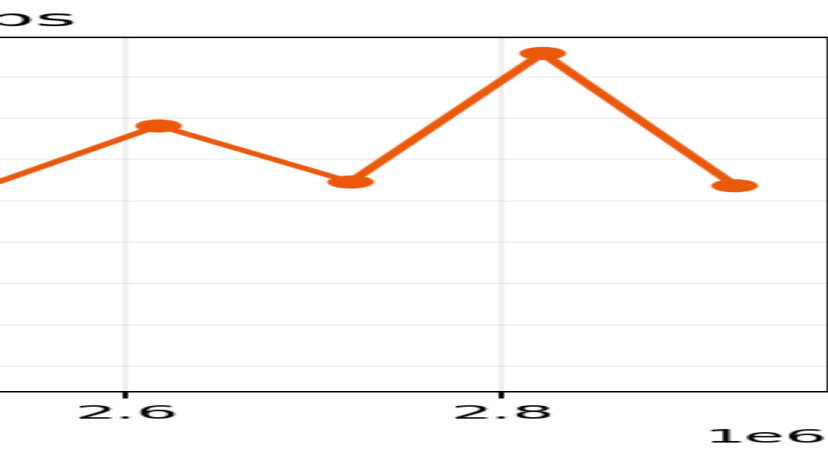
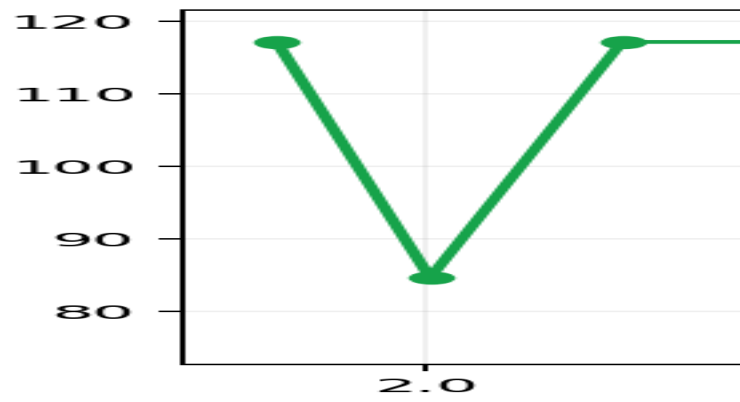
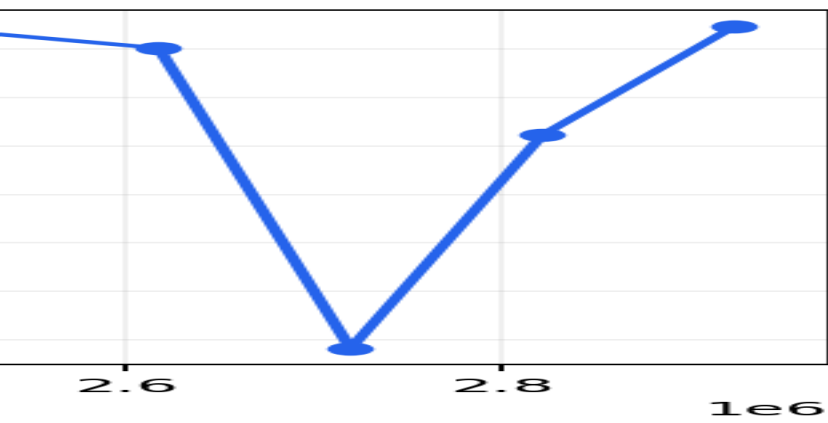
The overnight launcher script requested a longer run than the metadata preserved in **report/summary.json**: it asks for 80k phase-1 steps and 2.0M phase-2 steps. Because the report summary still lists 40k and 400k, the most reliable evidence of actual progress is the checkpoint record itself, which reaches generation 27 at 2,924,544 total timesteps.

Observed Outcomes

The stored checkpoint slice is clearly late-stage training rather than an entire run from scratch. It begins at generation 17 and ends at generation 27. Within this window, 7 of 11 checkpoint evaluations finished the course. The best raw progress came from generation 25 at 119.31 m, but that rollout still terminated off-track. The strongest successful checkpoint by reward is generation 27, which finished in 34.92 s with reward 116.63.

Summary





Across the recorded checkpoint evaluations in this artifact slice, the finish rate is 64%, mean finish time among successful checkpoints is 37.69 s, mean progress is 109.20 m, and mean reward is -188.27. Termination counts in the recorded slice are {'finish': 7, 'off_track': 2, 'stall': 1, 'timeout': 1}. That is sufficient to show the learned controller repeatedly completes the full course, while still exhibiting off-track, stall, and timeout failure modes at other checkpoints.

Artifacts And Deployment State

The RL pipeline generated more than scalar metrics. Each checkpoint refresh writes a live dashboard, reward and progress curves, section-success and failure histograms, plus a 3D replay snapshot. These artifacts are useful because they make it possible to inspect where a policy improved, not only whether the scalar reward happened to rise on a given generation.



The replay frame on the left is taken from the live snapshot bundle of the latest checkpoint. The figure on the right captures an important implementation detail uncovered during this supplement build: the exported policy manifest describes a 2-action network, which matches the direct-control training environment, but the runtime residual bot wrapper expects 3 outputs. In the present code this means the exported policy is considered incompatible and the runtime mixed controller falls back to the heuristic bot.

That does not invalidate the training work itself. The training environment, checkpoints, and final saved PPO file all show a functioning learned policy controller. It does, however, mean that the current playable runtime path is not yet demonstrating the learned policy directly. Closing that gap would require either exporting a residual 3-output policy or adapting the runtime integration to the 2-output direct-control policy.

What was undertaken: a PPO-based direct-control bot for the marble racer, trained through a staged curriculum and checkpointed with live qualitative evidence.

How it was done: 23-D observation vector, compact 64-64 MLP policy, checkpoint evaluation every 100k steps, and a snapshot pipeline that wrote plots and replay media to the monitor page.

What the outcomes were: late-stage checkpoints reached 2,924,544 timesteps, successful finishes were common in the recorded slice, the fastest logged finish was 31.37 s, and the best successful checkpoint was generation 27 with reward 116.63.

What remains to finish the feature: align export format and runtime controller expectations so the learned policy can drive the in-game bot path without falling back to the heuristic controller.